

RAPORT BADAWCZY: ARCHITEKTURA TECHNICZNA, STRATEGIA WIZUALNA I PROJEKT DOŚWIADCZEŃ UŻYTKOWNIKA DLA SYSTEMU "PROOF OF SUPPORT NFT" (ODZNAKA NFT)

1. Streszczenie Wykonawcze

W dobie cyfrowej ekonomii twórców, mechanizmy finansowania społecznościowego ewoluują z prostych transakcji pieniężnych w stronę złożonych systemów tożsamości cyfrowej i reputacji on-chain. Projekt "Proof of Support NFT" (Dowód Wsparcia) dla platformy TipJar+ stanowi awangardowe podejście do zagadnienia cyfrowego mecenatu, łącząc w sobie elementy finansowe, artystyczne oraz społecznościowe. Niniejszy raport stanowi wyczerpującą analizę techniczną i projektową, mającą na celu dostarczenie kompletnej specyfikacji dla implementacji niezwykłych tokenów (Soulbound Tokens - SBT), które służą jako trwałe świadectwo wsparcia udzielonego twórcom.

Centralnym założeniem projektu jest stworzenie systemu, który automatycznie generuje i dystrybuuje unikalne aktywa cyfrowe w odpowiedzi na transakcje napiwków (tipów). W przeciwieństwie do tradycyjnych NFT, które często są przedmiotem spekulacji rynkowej, "Proof of Support" ma charakter permanentny i niezwykły, co wymusza zastosowanie specyficznych standardów tokenizacji, takich jak ERC-5192. Wizualna reprezentacja tych tokenów opiera się na metaforze "słoika z napiwkami" (Tip Jar), który jest generowany proceduralnie przy użyciu zaawansowanych algorytmów graficznych (Canvas API) po stronie serwera. Każdy element graficzny – od poziomu wypełnienia słoika, przez jego estetykę, aż po unikalne wzory tła – jest deterministycznie powiązany z danymi transakcji, takimi jak kwota, data czy hash transakcji. Raport ten szczegółowo omawia architekturę smart kontraktów niezbędną do zapewnienia bezpieczeństwa i niezwykłości tokenów, strategię przechowywania metadanych w sieciach zdecentralizowanych (IPFS/Arweave) w celu zagwarantowania ich długowieczności, a także psychologię interfejsu użytkownika (UX/UI), ze szczególnym uwzględnieniem trybu ciemnego (Dark Mode) i mechanizmów natychmiastowej gratyfikacji ("pop-in"). Analiza uwzględnia najnowsze trendy w projektowaniu gier i aplikacji Web3 na rok 2025, kładąc nacisk na personalizację, dostępność oraz estetykę "premium".

2. Ewolucja Cyfrowego Mecenatu i Rola Tokenów Soulbound

2.1 Od Transakcji do Relacji: Nowy Paradygmat Web3

Tradycyjne platformy Web2, służące do wspierania twórców, opierają się na efemerycznych potwierdzeniach transakcji – e-mailach z podziękowaniami czy wpisach w bazie danych, które

znikają wraz z upadkiem platformy lub zamknięciem konta. W ekosystemie Web3, opartym na technologii blockchain, pojawia się możliwość przekształcenia jednorazowej wpłaty w trwały, cyfrowy artefakt. "Proof of Support NFT" nie jest jedynie pokwitowaniem; jest cyfrowym odpowiednikiem grawerowanej tabliczki na ławce w parku lub imienia fundatora wyrzeźbionego w cegle. Przenosi on ciężar relacji z poziomu "klient-usługodawca" na poziom "mecenasa-twórcy", budując trwałą historię wsparcia przypisaną do portfela użytkownika.

Koncepcja ta wpisuje się w szerszy trend "społecznego sygnalizowania" (social signaling), gdzie posiadane aktywa cyfrowe świadczą o wartościach, zainteresowaniach i statusie społecznym użytkownika. W przypadku TipJar+, NFT pełni funkcję dowodu na wczesne wsparcie (early support) lub wysoki poziom zaangażowania finansowego, co w przyszłości może być wykorzystywane przez twórców do nagradzania lojalnych fanów (token-gating).

2.2 Definicja i Mechanika Tokenów Soulbound (SBT)

Kluczowym wymogiem projektowym jest niezbywalność tokena. W standardowym ekosystemie NFT (ERC-721), tokeny są z definicji dobrami zbywalnymi, co w kontekście "dowodu wsparcia" prowadziłoby do patologii rynkowych – możliwości sprzedaży "reputacji" bycia wspierającym. Aby temu zapobiec, konieczne jest wdrożenie mechanizmu "Soulbound". Terminu tego użył po raz pierwszy Vitalik Buterin, nawiązując do przedmiotów z gry World of Warcraft, które po podniesieniu zostają na stałe przypisane do gracza.

W kontekście TipJar+, SBT gwarantuje, że:

1. Odznaka nie może zostać sprzedana na rynku wtórnym (OpenSea, Blur).
2. Historia wsparcia jest nierozdzielnie związana z adresem, który dokonał transakcji.
3. Wartość tokena jest czysto reputacyjna i sentymentalna, a nie spekulacyjna.

Implementacja tej funkcjonalności wymaga wyboru odpowiedniego standardu technicznego, który zostanie omówiony w kolejnych rozdziałach, ze szczególnym uwzględnieniem różnic między prostym blokowaniem transferów a pełną implementacją standardu ERC-5192.

3. Architektura Techniczna i Standardy Smart Kontraktów

Wybór odpowiedniego standardu tokena jest decyzją krytyczną, determinującą kompatybilność z portfelami, indeksatorami oraz przyszłymi integracjami w ekosystemie Ethereum i EVM.

3.1 Analiza Porównawcza: ERC-721 vs. ERC-5192

Chociaż standard ERC-721 jest fundamentem rynku NFT, jego domyślna implementacja zakłada pełną zbywalność aktywów. Istnieją dwie główne szkoły tworzenia tokenów SBT: modyfikacja ERC-721 oraz dedykowany standard ERC-5192.

3.1.1 Podejście Naiwne: Modyfikowany ERC-721

Najprostszym sposobem na stworzenie SBT jest wzięcie standardowego kontraktu ERC-721 (np. od OpenZeppelin) i nadpisanie funkcji transferowych (transferFrom, safeTransferFrom), tak aby zawsze zwracały błąd (revert) przy próbie wywołania.

Zalety:

- Szybkość wdrożenia.

- Częściowa kompatybilność z narzędziami oczekującymi interfejsu ERC-721.

Wady:

- **Brak transparentności:** Zewnętrzne systemy (np. OpenSea) nie wiedzą, że token jest niezbywalny, dopóki transakcja nie zakończy się niepowodzeniem. Prowadzi to do frustracji użytkowników, którzy mogą próbować wystawić token na sprzedaż, tracąc opłaty transakcyjne (gas).
- **Martwy kod (Dead Code):** Kontrakt zawiera logikę zatwierdzania operatorów (approve, setApprovalForAll), która w przypadku SBT jest całkowicie zbędna, zwiększając rozmiar bajtkodu i koszt wdrożenia.

3.1.2 Rekomendowane Rozwiązanie: Standard ERC-5192

ERC-5192 ("Minimal Soulbound NFTs") to oficjalny standard zaprojektowany specjalnie w celu rozwiązania problemów modyfikowanych ERC-721. Rozszerza on EIP-721 o mechanizm detekcji statusu blokady.

Kluczowe elementy implementacji dla TipJar+:

1. **Interface ID:** Kontrakt musi implementować interfejs ERC-165, zwracając true dla identyfikatora ERC-5192. Pozwala to portfelom i marketplacem na wyświetlanie ikony kłódki lub ukrywanie przycisku "Sprzedaj".
2. **Zdarzenia (Events):** Emisja zdarzenia Locked(uint256 tokenId) natychmiast po wybiciu tokena (mintingu). Indeksatory blockchain (np. The Graph) mogą dzięki temu natychmiastowo skatalogować token jako SBT.
3. **Funkcja locke[span_7](start_span)[span_7](end_span)d():** Publiczna funkcja view, która dla każdego tokena TipJar+ zwróci wartość true.

Poniższa tabela przedstawia porównanie funkcjonalności w kontekście wymagań projektu:

Cecha	Zmodyfikowany ERC-721	ERC-5192 (Rekomendowany)
Niezbywalność	Tak (przez błąd transakcji)	Tak (przez standard)
Detekcja przez UI	Nieemożliwa/Trudna	Automatyczna (EIP-165)
Koszt Gasu	Wyższy (zbędny kod)	Zoptymalizowany
Kompatybilność	Ryzyko błędów na giełdach	Pełne wsparcie narzędziowe
Semantyka	"Zepsuty" token zbywalny	Prawdziwy token tożsamości

3.2 Logika Smart Kontraktu "TipJarSBT"

Rekomendowana architektura kontraktu powinna dziedziczyć z biblioteki OpenZeppelin dla bezpieczeństwa, ale implementować interfejs IERC5192.

Pseudokod struktury kontraktu:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "../interfaces/IERC5192.sol";

contract TipJarSBT is ERC721, IERC5192 {
    // Konstruktor ustawiający nazwę i symbol
    constructor() ERC721("TipJar Proof of Support", "TIP") {}

    // Funkcja mintująca, dostępna tylko dla autoryzowanego serwera
```

```

(MINTER_ROLE)
    function safeMint(address to, string memory uri) public onlyMinter
    {
        uint256 tokenId = _tokenIdCounter.current();
        _tokenIdCounter.increment();
        _safeMint(to, tokenId);
        _setTokenURI(tokenId, uri);

        // Kluczowe dla ERC-5192: Emisja zdarzenia blokady
        emit Locked(tokenId);
    }

    // Nadpisanie funkcji transferu, aby uniemożliwić przenoszenie
    function _beforeTokenTransfer(address from, address to, uint256
tokenId, uint256 batchSize) internal override {
        require(from == address(0) |

| to == address(0), "Err: Token is Soulbound");
        super._beforeTokenTransfer(from, to, tokenId, batchSize);
    }

    // Implementacja interfejsu ERC-5192
    function locked(uint256 tokenId) external view override returns
(bool) {
        return true; // Zawsze zablokowany
    }

    // Obsługa EIP-165
    function supportsInterface(bytes4 interfaceId) public view
override(ERC721, IERC165) returns (bool) {
        return interfaceId == type(IERC5192).interfaceId |

| super.supportsInterface(interfaceId);
    }
}

```

Powyższa implementacja gwarantuje, że token zachowuje się jak standardowy NFT w kwestii wyświetlania w portfelu (dzięki dziedziczeniu ERC-721), ale technicznie blokuje każdą próbę transferu inną niż mintowanie (tworzenie) lub spalanie (niszczenie).

4. Strategia Generowania Grafiki i Wizualizacja Danych

Wizualna atrakcyjność NFT jest kluczowa dla satysfakcji użytkownika. Zgodnie z wymaganiem, grafika musi być generowana automatycznie (proceduralnie) i zawierać elementy dynamiczne zależne od danych transakcji.

4.1 Generowanie po Stronie Serwera (Server-Side Rendering)

Ze względów bezpieczeństwa i spójności, generowanie grafiki nie może odbywać się w przeglądarce użytkownika (client-side). Użytkownik mógłby zmanipulować kod JS, aby wygenerować "Legendarny" słoik przy minimalnym napiwku. Proces ten musi odbywać się w bezpiecznym środowisku serwerowym (Node.js).

Stos technologiczny: Do generowania grafiki rekomendowane jest użycie biblioteki **node-canvas**. Jest ona implementacją API Canvas (znanego z HTML5) działającą w środowisku Node.js.

- **Dlaczego nie Sharp?** Biblioteka Sharp jest doskonała do zmiany rozmiaru i prostego nakładania obrazów, ale node-canvas oferuje znacznie większą elastyczność w *rysowaniu* kształtów, tekstu i generowaniu złożonych wzorów geometrycznych wymaganych przez "unikalny wzór generowany z hasha".

4.2 System Warstw i Kompozycja Obrazu (1024x1024px)

Aby uzyskać efekt "Digital art, minimalistyczny" z elementami 3D (słoik), system musi składać obraz z wielu warstw (layers), nakładanych w określonej kolejności (Z-index).

Struktura Warstw (od dołu do góry):

1. **Tło (Background):**
 - Ciemne, gradientowe tło zgodne z estetyką "Dark Mode".
 - Kolorystyka tła może delikatnie zmieniać się w zależności od pory dnia wsparcia (np. ciemniejszy fiolet w nocy, jaśniejszy granat w dzień).
2. **Unikalny Wzór Hasha (Hash Pattern):**
 - Na tle rysowany jest subtelny, geometryczny wzór (np. diagram Voronoia lub fale Perlin Noise).
 - **Algorytm:** Hash transakcji (np. 0x3a...) jest używany jako ziarno (seed) dla generatora liczb pseudolosowych. Dzięki temu dla tej samej transakcji zawsze wygeneruje się ten sam wzór, ale każda inna transakcja da unikalny wynik. Stanowi to wizualny "odcisk palca" transakcji.
3. **Tylna Ściana Słoika (Jar Back):**
 - Półprzezroczysta warstwa szkła, zapewniająca głębię.
4. **Zawartość (Fill Layer):**
 - To kluczowy element dynamiczny. Poziom wypełnienia słoika monetami zależy od kwoty napiwku.
 - Logika: $\text{fillHeight} = (\text{amount} / \text{maxBaseline}) * \text{jarHeight}$.
 - Rodzaj monet: Złote monety z logotypem waluty (np. ETH, USDC) lub abstrakcyjne, świecące kule energii dla estetyki magicznej.
5. **Przednia Ściana Słoika (Jar Front & Highlights):**
 - Refleksy świetlne, odbicia i kontury słoika. Ta warstwa nadaje obiektowi trójwymiarowości i "szklanego" charakteru.
6. **Etykieta i Typografia (Metadata Layer):**
 - Na słoiku "naklejona" jest etykieta z nazwą twórcy i datą.
 - Użycie czcionek o wysokiej czytelności, ale z charakterem (np. szeryfowe dla nazwy twórcy, monospace dla daty/hasza). Tekst powinien być lekko zniekształcony (curved text), aby symulować krzywiznę słoika.
7. **System Rzadkości (Wreath/Aura):**

- Dla wysokich kwot wsparcia, wokół słoika renderowany jest dodatkowy element ozdobny (wieniec, poświata).

4.3 Psychologia Kolorów i System Rzadkości (Rarity Tiers)

Zgodnie z wymaganiami, NFT musi wizualnie odzwierciedlać wartość wsparcia. W świecie gamingu i Web3 utrwalił się konkretny kod kolorystyczny oznaczający rzadkość przedmiotów, wywodzący się z gier RPG (jak World of Warcraft czy Diablo). Zastosowanie tego systemu jest intuicyjne dla użytkowników i natychmiast komunikuje prestiż odznaki.

Tabela Rzadkości dla TipJar+:

Poziom (Tier)	Zakres Kwotowy (Przykładowy)	Kolor Dominujący	Element Wizualny (Wieniec/Akcent)	Hex Code (Dark Mode)
Braźowy (Common)	\$1 - \$10	Szary / Braźowy	Prosty słoik, brak aury	#B0C4DE / #CD7F32
Srebrny (Uncommon)	\$10 - \$50	Zielony / Turkusowy	Delikatna poświata u podstawy	#36A90C / #2AC9D2
Złoty (Rare)	\$50 - \$100	Niebieski / Szafir	Niebieski płomień wewnątrz	#0070DD
Platynowy (Epic)	\$100 - \$500	Fioletowy	Ozdobny wieniec laurowy	#A335EE
Diamantowy (Legendary)	> \$500	Pomarańczowy / Złoty	Złota aura, cząsteczki (particles)	#FF8000 / #FFD700

Uzasadnienie: Kolory w trybie Dark Mode muszą być nasycone i jasne, aby kontrastowały z ciemnym tłem (#121212). Unikamy czystej bieli i czystej czerni na rzecz odcieni szarości, aby zmniejszyć zmęczenie oczu.

5. Infrastruktura Metadanych i Bezpieczeństwo Przechowywania

Metadane są sercem NFT – to w nich zapisane są informacje o twórcy, dacie i wiadomości. Sposób ich przechowywania determinuje trwałość odznaki.

5.1 Struktura JSON i Standard OpenSea

Aby NFT wyświetlało się poprawnie na OpenSea, MetaMask i innych platformach, plik metadanych musi być zgodny ze specyfikacją ERC-721 Metadata Standard. Należy zwrócić szczególną uwagę na atrybut `display_type`, który pozwala na poprawne formatowanie dat i liczb.

Przykład pliku metadata.json dla TipJar+:

```
{
  "name": "Odznaka Wsparcia: @Kryptonim",
  "description": "Niezbywalny dowód wsparcia dla twórcy @Kryptonim. Wybito przez TipJar+.",
  "image": "ipfs://bafybeig.../image.png",
  "external_url": "https://tipjar.plus/tx/0x123...",
  "attributes":
```

}

Kwestia Wiadomości: Przechowywanie wiadomości od użytkownika bezpośrednio w metadanych (JSON) jest ryzykowne ze względu na brak możliwości edycji (immutability). Jeśli użytkownik wpisze dane wrażliwe lub obraźliwe, pozostaną one tam na zawsze. Rekomenduje się filtrowanie wiadomości przed mintowaniem (moderacja AI) lub zapisywanie wiadomości tylko w bazie danych off-chain, a w metadanych umieszczanie jedynie hasha wiadomości dla weryfikacji integralności. W niniejszym projekcie przyjmujemy założenie filtrowania pre-minting.

5.2 Strategia Przechowywania: IPFS vs. Arweave

Wymaganie projektowe sugeruje użycie IPFS lub Arweave.

- **IPFS (InterPlanetary File System):** Standard rynkowy. Pliki są adresowane po treści (Content Addressing - CID). Jednak IPFS nie gwarantuje trwałości sam z siebie – pliki muszą być "przypięte" (pinned) przez węzeł. Jeśli TipJar+ przestanie opłacać pinning service (np. Pinata), obrazki mogą zniknąć.
- **Arweave:** Oferuje model "płać raz, przechowuj na zawsze". Jest to rozwiązanie idealne dla tokenów Soulbound, które z założenia mają trwać wiecznie. Koszt jest nieco wyższy na początku, ale eliminuje koszty cykliczne i ryzyko utraty danych.

Rekomendacja: Zastosowanie hybrydy. Główny obraz i metadane powinny trafić na **Arweave** dla absolutnej trwałości. W kodzie kontraktu tokenURI powinno wskazywać na bramkę Arweave, ale z możliwością fallbacku do IPFS dla redundancji.

6. Projekt Interfejsu (UI) i Doświadczenie Użytkownika (UX)

Interfejs użytkownika musi być spójny z estetyką premium i "Dark Mode", zapewniając jednocześnie natychmiastową informację zwrotną.

6.1 Miniatura (Thumbnail) i Listy

- **Rozmiar:** 256x256px.
- **Kształt:** Zaokrąglony kwadrat (border-radius: 12-16px). Idealne koło (circle) może przycinać istotne detale generowanego słoika w rogach.
- **Obramowanie (Border):** Zastosowanie 2-pikselowego obramowania w kolorze rzadkości (np. złoty dla "Złotego Mecenasa"). Pozwala to na szybkie skanowanie wzrokiem listy wsparć i identyfikację najważniejszych donatorów bez wchodzenia w szczegóły.

6.2 Modal Szczegółów NFT: Anatomia "Trophy Case"

Modal to moment celebracji. Nie jest to zwykłe okno informacyjne, lecz cyfrowa gablota z trofeum.

Projekt Wizualny (Dark Mode):

- **Tło Modala:** Ciemnoszary (#1F1F1F lub #2D2D2D) zamiast czystej czerni, aby zbudować głębię.
- **Podgląd Grafiki:** Centralnie umieszczony obraz 512x512px (skalowany z 1024px) z delikatnym cieniem (box-shadow) w kolorze rzadkości, symulującym poświatę (glow

effect).

- **Karta Metadanych:** Poniżej grafiki, sekcja z danymi ułożona w siatce. Etykiety (Label) w kolorze szarym, wartości (Value) w bieli. Hash transakcji skrócony (np. 0x3a...4f9) z ikoną kopiowania.

Interakcje i Przyciski (Call to Action):

1. **"Udostępnij na Twitterze":** Przycisk priorytetowy. Nie może generować zwykłego linku. Powinien korzystać z dynamicznie generowanego obrazu Open Graph (OG Image), który łączy grafikę NFT z tekstem "Wsparłem!". To kluczowy mechanizm wirusowości platformy.
2. **"Pobierz Obraz":** Pozwala użytkownikowi zapisać plik PNG na dysku.
3. **"Zobacz na Explorze":** Link zewnętrzny (Etherscan/Arbiscan) dla weryfikacji on-chain.

6.3 Animacja "Pop-in" i Optimistic UI

Transakcje blockchainowe trwają (od kilku sekund na L2 do minut na Mainnetcie). Użytkownik nie może czekać na potwierdzenie bloku, aby zobaczyć efekt. Należy zastosować wzorzec

Optimistic UI.

1. **Trigger:** Użytkownik klika "Wyślij Napiwek".
2. **Animacja Wstępna:** Natychmiastowa eksplozja konfetti (biblioteka react-confetti-explosion) w kolorach zgodnych z rzadkością (np. złote konfetti dla dużego napiwku).
3. **Pop-in:** Modal pojawia się z animacją wejścia (skalowanie od 0.8 do 1.0 + fade in).
4. **Stan Przejściowy:** Zamiast finalnego NFT, początkowo wyświetla się animowany "Słoik w budowie" lub szkielet (skeleton loader) z napisem "Wybijanie Odznaki...".
5. **Finalizacja:** Gdy backend potwierdzi wygenerowanie grafiki (nawet przed pełnym potwierdzeniem bloku), obraz podmienia się na właściwy NFT.

7. Implementacja i Wymagania Techniczne - Podsumowanie

Aby zrealizować wizję "Proof of Support", deweloperzy muszą zintegrować następujące komponenty:

Komponent	Technologia / Standard	Uwagi Implementacyjne
Smart Kontrakt	Solidity, ERC-5192	Dziedziczenie ERC-721, implementacja interfejsu locked(), emisja zdarzeń.
Generator Grafiki	Node.js + node-canvas	Użycie Transaction Hash jako ziarna (seed) dla deterministycznego generowania wzorów.
Storage	Arweave (rekomendowany)	Zapewnienie permentancji danych. Fallback do IPFS.
Backend	Node.js / Python	Nasłuchiwanie zdarzeń wpłat, kolejkovanie generowania, obsługa meta-transakcji (gasless minting).
Frontend	React / Vue	Obsługa portfeli

Komponent	Technologia / Standard	Uwagi Implementacyjne
		(Wagmi/Ethers.js), animacje CSS/JS, tryb Dark Mode.

8. Wnioski Końcowe

Projekt "Proof of Support NFT" dla TipJar+ to coś więcej niż techniczna implementacja standardu ERC-5192. To starannie zaprojektowany system psychologiczny, który wykorzystuje mechanizmy rzadkości, estetykę kolekcjonerską i trwałość blockchained, aby wzmocnić więź między twórcą a fanem. Poprzez zastosowanie generatywnej sztuki opartej na danych transakcyjnych, każdy token staje się unikalnym dziełem sztuki, a nie generycznym pokwitowaniem. Zastosowanie standardu Soulbound eliminuje spekulację finansową, skupiając wartość na reputacji i historii wsparcia. Przedstawiona architektura zapewnia skalowalność, bezpieczeństwo i wirusowy potencjał marketingowy, niezbędny dla sukcesu nowoczesnej platformy monetyzacji twórców.

Cytowane prace

1. Reference implementation of ERC5192 Minimal Soulbound Tokens - GitHub, <https://github.com/attestate/ERC5192>
2. EIP-5192 - EIPs Insights, <https://eipsinsight.com/eips/eip-5192>
3. What is Dynamic NFT? How dNFTs Work, Examples, Pros & Cons | Cube Exchange, <https://www.cube.exchange/what-is/dynamic-nft>
4. Soulbound NFTs: Should they be a separate standard instead of ERC-721 extension?, https://www.reddit.com/r/ethdev/comments/1q8mp6u/soulbound_nfts_should_they_be_a_separate_standard/
5. Reference implementation of ERC5192 Minimal Soulbound Tokens : r/ethdev - Reddit, https://www.reddit.com/r/ethdev/comments/10asl4a/reference_implementation_of_erc5192_minimal/
6. ERC-721 - OpenZeppelin Docs, <https://docs.openzeppelin.com/contracts/5.x/erc721>
7. ERC721 - OpenZeppelin Docs, <https://docs.openzeppelin.com/contracts/4.x/api/token/erc721>
8. Generative Art with Node.js and Canvas - Matt DesLauriers, <https://mattdesl.svbtle.com/generative-art-with-nodejs-and-canvas>
9. Which library should I use for server-side image manipulation on Node.JS? - Stack Overflow, <https://stackoverflow.com/questions/10692075/which-library-should-i-use-for-server-side-image-manipulation-on-node-js>
10. Generative Art with JavaScript and Canvas: A Beginner's Playground - DEV Community, https://dev.to/shayo_victor_c02f1777210e/generative-art-with-javascript-and-canvas-a-beginners-playground-2cb8
11. Generative Art Algorithms: How to Build an NFT Collection - Surge Women, <https://www.surgewomen.io/dev-transition-into-web3/generative-art-algorithms-how-to-build-an-nft-collection>
12. 7 Generative Art NFTs Redefining Digital Creativity | OpenSea, <https://opensea.io/blog/articles/generative-art-nfts>
13. Magic Jar: Over 12829 Royalty-Free Licensable Stock Illustrations & Drawings, https://www.shutterstock.com/search/magic-jar?image_type=illustration
14. Magical Jar Illustrations & Vectors - Dreamstime.com, <https://www.dreamstime.com/illustration/magical-jar.html>
15. Should I use "conventional" colors to represent item rarity?, <https://gamedev.stackexchange.com/questions/140586/should-i-use-conventional-colors-to-represent-item-rarity>
16. Item Rarity Color Palette, <https://www.color-hex.com/color-palette/38466>

17. Dark mode UI design: Best practices and examples - LogRocket Blog,
<https://blog.logrocket.com/ux-design/dark-mode-ui-design-best-practices-and-examples/> 18.
Rarity Color Scheme - Palettes - SchemeColor.com, <https://www.schemecolor.com/rarity.php> 19.
Metadata Standards - OpenSea Docs, <https://docs.opensea.io/docs/metadata-standards> 20.
NFT metadata standards - Tableland Docs,
<https://docs.tableland.xyz/playbooks/concepts/nft-metadata> 21. NFT Metadata: Storage &
Formatting Best Practices - Fleek.xyz,
<https://fleek.xyz/guides/storing-nft-metadata-and-standards/> 22. Best Practices for Storing NFT
Data using IPFS, <https://docs.ipfs.tech/how-to/best-practices-for-nft-data/> 23. Dark Mode NFT
app Part 5 - List - UI Design Quick Apps in Figma - Design+Code,
<https://designcode.io/quick-apps-figma-nft-list/> 24. Nft Social Media - Etsy,
https://www.etsy.com/market/nft_social_media 25. confetti-explosion-react - Yarn Classic,
<https://classic.yarnpkg.com/en/package/confetti-explosion-react> 26. react-confetti-boom - NPM,
<https://npmjs.com/package/react-confetti-boom> 27. CSS Shake Animation | UnusedCSS,
<https://unused-css.com/blog/css-shake-animation/> 28. Animate.css | A cross-browser library of
CSS animations., <https://animate.style/>